scripts/validate-jackett-sidecar.sh

Purpose. End-to-end readiness check for the Jackett Torznab sidecar (and the optional FlareSolverr Cloudflare-challenge solver) in the Lava local stack. Brings the sidecar up via `tools/lava-containers/docker-compose.jackett.yml`, waits for the compose healthchecks to report HEALTHY, then independently re-probes the user-visible surfaces from the host and prints PASS/FAIL with verbatim responses. Always tears the sidecar down on exit (unless `--keep`).

Constitutional basis. §6.B ("Up" is not application-healthy — the probe is a Torznab `t=caps` request + a FlareSolverr `sessions.list` POST, not a TCP check), §6.J anti-bluff (it re-runs the real surface from the host and prints verbatim output rather than trusting container State), §6.R (ports/host/api_key from `.env`, never literals), §6.H (the api_key is never printed — the logged URL is redacted), §6.U (no sudo), §6.T.2 (read-only curls + one short-lived sidecar; no Gradle, no emulator, no build).

Usage

```
# Validate Jackett only (RuTracker / RuTor / NNMClub indexers — no
        Cloudflare):
scripts/validate-jackett-sidecar.sh

# Validate Jackett + FlareSolverr (the IPTorrents / Cloudflare
        path):
scripts/validate-jackett-sidecar.sh --cloudflare

# Leave the sidecar running after the probe (debugging):
scripts/validate-jackett-sidecar.sh --keep

# Force a runtime instead of auto-detect (podman is preferred):
scripts/validate-jackett-sidecar.sh --runtime docker
```

What it checks

Probe	Surface	PASS condition
1		

Probe	Surface	PASS condition
	Jackett Torznab caps	`GET .../results/torznab/api?t=caps&apikey=…` → HTTP 200 + parseable Torznab XML, api_key NOT rejected
2	FlareSolverr liveness	`POST /v1 {"cmd":"sessions.list"}` → HTTP 200 + `"status":"ok"` (only with `--cloudflare`)

The in-container compose healthcheck is the same `t=caps` probe; the host-side re-probe additionally proves the published loopback port is reachable by a host-net peer (which is how `lava-api-go` — `network_mode: host` — reaches it).

Exit codes

Code	Meaning
0	all probed surfaces PASS — sidecar READY for lava-api-go
1	at least one probe FAILED (logs tailed above)
2	config / usage error (no runtime, missing api_key, bad arg)

Preconditions (the manual, non-automatable part)

Jackett indexer config is **stateful**, persisted in the gitignored `/config` volume (`LAVA_JACKETT_CONFIG_DIR`). Two things cannot be fully automated:

1. **api_key generation.** Jackett writes a fresh api_key into `ServerConfig.json` on first run. Copy it into `.env` as `LAVA_API_JACKETT_APIKEY` before running this script (it refuses the placeholder value).
2. **Indexer add (per tracker).** Adding IPTorrents (with the FlareSolverr URL) or any tracker is a one-time WebUI/API step. The caps probe for `all` returns 200 with an empty `<caps>` on a zero-indexer config — accepted as "surface live + api_key valid" — but a real search needs at least one indexer added.

See `docs/qa/jackett-sidecar-deployment.md` for the full one-time setup.

§11.4.74 catalogue note

Container lifecycle reuses the project's existing `podman/docker compose` runtime detection pattern (same as `run-chaos-stress.sh`, `sonar-scan.sh`); the script adds only the Jackett-specific bring-up + Torznab/FlareSolverr probe glue, reimplementing no generic capability.